

IK141 Struktur Data
Pendahuluan Struktur Data
Review Array Fungsi, dan Prosedur



Di Susun Oleh :
Deden Ahmad Jamil
2501518

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER
FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS PENDIDIKAN INDONESIA

10 Februari 2026

1. Implementasi dan Hasil

Implementasi dan Hasil

ss code program no1:

```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 // Definisi struktur data pasien (Linked List)
9 struct Node {
10     int id;
11     char nama[50];
12     char penyakit[50];
13     struct Node *next;
14 };
15
16 struct Node *head = NULL;
17
```



```
1 // Menampilkan semua data dari awal sampai akhir
2 void tampilkan() {
3     struct Node *bantu = head;
4     printf("\nData Antrian Pasien:\n");
5     printf("===== \n");
6     while (bantu != NULL) {
7         printf("ID: %d | Nama: %s | Penyakit: %s\n", bantu->id, bantu->nama, bantu->penyakit);
8         bantu = bantu->next; // Geser ke node berikutnya
9     }
10    printf("===== \n");
11 }
```



```
1 // Menambah data di urutan paling depan
2 void insertAwal(int id, char nama[], char penyakit[]) {
3     struct Node *baru = (struct Node*)malloc(sizeof(struct Node));
4     baru->id = id;
5     strcpy(baru->nama, nama);
6     strcpy(baru->penyakit, penyakit);
7
8     baru->next = head; // Sambungkan ke head lama
9     head = baru;      // Jadikan node baru sebagai head
10 }
```



```
1 // Menambah data di urutan paling belakang
2 void insertAkhir(int id, char nama[], char penyakit[]) {
3     struct Node *baru = (struct Node*)malloc(sizeof(struct Node));
4     baru->id = id;
5     strcpy(baru->nama, nama);
6     strcpy(baru->penyakit, penyakit);
7     baru->next = NULL;
8
9     if (head == NULL) {
10         head = baru;    // Jika kosong, langsung jadi head
11     } else {
12         struct Node *bantu = head;
13         while (bantu->next != NULL) {
14             bantu = bantu->next; // Cari node terakhir
15         }
16         bantu->next = baru;    // Hubungkan node terakhir ke node baru
17     }
18 }
```



```
1 // nama : deden ahamd jamil
2 // nim  : 2501518
3
4 #include "antrian_rs.h"
5
6 int main() {
7     // Memasukan data langsung
8     insertAwal(1, "Ryan", "Demam");
9     insertAkhir(2, "Reybano", "Batuk");
10    insertAkhir(3, "Akbar", "Flu");
11
12    // Menampilkan hasil
13    tampilkan();
14
15    return 0;
16 }
```

ss output nomor1:

```
dedeng@Deden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP4-linkedlist-part2/no1
$ ./LAT2_A_NAMA_2501518.exe

Data Antrian Pasien:
=====
ID: 1 | Nama: Ryan | Penyakit: Demam
ID: 2 | Nama: Reybano | Penyakit: Batuk
ID: 3 | Nama: Akbar | Penyakit: Flu
=====
```

Ss code program no 2:

```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 // Definisi struktur data pasien (Linked List)
9 struct Node {
10     int id;
11     char nama[50];
12     char penyakit[50];
13     struct Node *next;
14 };
15
16 struct Node *head = NULL;
```



```
1 void insertTengahSebelum(int idCari, int idBaru, char namaBaru[], char penyakitBaru[]) {
2     if (head == NULL) return; // Keluar kalau list masih kosong
3
4     // Kalau ternyata yang dicari itu data pertama
5     if (head->id == idCari) {
6         insertAwal(idBaru, namaBaru, penyakitBaru);
7         return;
8     }
9
10    struct Node *bantu = head;
11    // Cari posisi berhenti tepat di depan node yang dituju
12    while (bantu->next != NULL && bantu->next->id != idCari) {
13        bantu = bantu->next;
14    }
15
16    // Kalau idCari ketemu (nggak sampai ujung list)
17    if (bantu->next != NULL) {
18        struct Node *baru = (struct Node*)malloc(sizeof(struct Node));
19        baru->id = idBaru;
20        strcpy(baru->nama, namaBaru);
21        strcpy(baru->penyakit, penyakitBaru);
22
23        // Sambungin pointer: baru ke depan, bantu ke baru
24        baru->next = bantu->next;
25        bantu->next = baru;
26    }
27 }
```



```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include "antrian_rs.h"
5
6 int main() {
7     // jawaban no 1
8     insertAwal(1, "Ryan", "Demam");
9     insertAkhir(2, "Reybano", "Batuk");
10    insertAkhir(3, "Akbar", "Flu");
11
12    // jawaban no 2
13    insertAwal(0, "Redit", "Sesak Napas");
14    insertTengahSebelum(3, 4, "Sasa", "Sakit Kepala");
15
16    // Menampilkan hasil
17    tampilkan();
18
19    return 0;
20 }
```

Ss output nomor 2:

```
deden@Dden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP4-linkedlist-part2/no2
$ ./LAT2_A_NAMA_2501518.exe
```

Data Antrian Pasien:

```
=====
ID: 0 | Nama: Radit | Penyakit: Sesak Napas
ID: 1 | Nama: Ryan | Penyakit: Demam
ID: 2 | Nama: Reybano | Penyakit: Batuk
ID: 4 | Nama: Sasa | Penyakit: Sakit Kepala
ID: 3 | Nama: Akbar | Penyakit: Flu
=====
```

Ss code program no 3:



```
1  // nama : deden ahmad jamil
2  // nim : 2501518
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7
8  // Definisi struktur data pasien (Linked List)
9  struct Node {
10     int id;
11     char nama[50];
12     char penyakit[50];
13     struct Node *next;
14 };
15
16 struct Node *head = NULL;
```



```
1 // Update penyakit pasien kalau ID-nya cocok
2 void updatePenyakit(int idCari, char penyakitBaru[]) {
3     struct Node *bantu = head;
4     while (bantu != NULL) {
5         if (bantu->id == idCari) {
6             strcpy(bantu->penyakit, penyakitBaru); // Timpa penyakit lama
7             printf("Data pasien %s berhasil diperbarui.\n", bantu->nama);
8             return;
9         }
10        bantu = bantu->next;
11    }
12 }
```



```
1 // Tambah data setelah ID tertentu
2 void insertTengahSesudah(int idCari, int idBaru, char namaBaru[], char penyakitBaru[]) {
3     struct Node *bantu = head;
4
5     // Cari ID yang dimaksud
6     while (bantu != NULL && bantu->id != idCari) {
7         bantu = bantu->next;
8     }
9
10    // Kalau ketemu, selipkan di depannya
11    if (bantu != NULL) {
12        struct Node *baru = (struct Node*)malloc(sizeof(struct Node));
13        baru->id = idBaru;
14        strcpy(baru->nama, namaBaru);
15        strcpy(baru->penyakit, penyakitBaru);
16
17        baru->next = bantu->next; // Baru nunjuk ke temannya bantu
18        bantu->next = baru;      // Bantu sekarang nunjuk ke baru
19    }
20 }
```



```
1 // Buang data pertama
2 void hapusAwal() {
3     if (head != NULL) {
4         struct Node *temp = head; // Pegang dulu biar nggak hilang
5         head = head->next;         // Pindahin head ke orang kedua
6         free(temp);                // Hapus data yang tadi dipegang
7     }
8 }
```



```
1 // Buang data paling ujung
2 void hapusAkhir() {
3     if (head == NULL) return; // Kalau kosong ya nggak ada yang dihapus
4     // Kalau cuma sendirian, langsung hapus aja
5     if (head->next == NULL) {
6         free(head);
7         head = NULL;
8         return;
9     }
10    struct Node *bantu = head;
11    // Cari satu orang sebelum orang terakhir
12    while (bantu->next->next != NULL) {
13        bantu = bantu->next;
14    }
15    free(bantu->next); // Hapus yang paling belakang
16    bantu->next = NULL; // Putus biar nggak nyari yang udah nggak ada
17 }
```



```
1 // nama : deden ahamd jamil
2 // nim : 2501518
3
4 #include "antrian_rs.h"
5
6 int main() {
7     // jawaban no 1
8     insertAwal(1, "Ryan", "Demam");
9     insertAkhir(2, "Reybano", "Batuk");
10    insertAkhir(3, "Akbar", "Flu");
11
12    // jawaban no 2
13    insertAwal(0, "Redit", "Sesak Napas");
14    insertTengahSebelum(3, 4, "Sasa", "Sakit Kepala");
15
16    // jawaban no 3
17    updatePenyakit(1, "Tipes");
18    insertTengahSesudah(4, 5, "Aura", "Maag");
19    hapusAwal();
20    hapusAkhir();
21
22    // Menampilkan hasil
23    tampilkan();
24
25    return 0;
26 }
```

Ss output nomor 3:

```
deden@Dden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP4-linkedlist-part2/no3
$ ./LAT2_A_NAMA_2501518.exe
Data pasien Ryan berhasil diperbarui.

Data Antrian Pasien:
=====
ID: 1 | Nama: Ryan | Penyakit: Tipes
ID: 2 | Nama: Reybano | Penyakit: Batuk
ID: 4 | Nama: Sasa | Penyakit: Sakit Kepala
ID: 5 | Nama: Aura | Penyakit: Maag
=====
```

Ss code program nomor 4:



```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 // Definisi struktur data pasien (Linked List)
9 struct Node {
10     int id;
11     char nama[50];
12     char penyakit[50];
13     struct Node *next;
14 };
15
16 struct Node *head = NULL;
```



```
1 // Ganti nama pasien berdasarkan ID-nya
2 void updateNama(int idCari, char namaBaru[]) {
3     struct Node *bantu = head;
4     while (bantu != NULL) {
5         if (bantu->id == idCari) {
6             strcpy(bantu->nama, namaBaru); // Timpa nama lama dengan yang baru
7             printf("Data pasien dengan id %d berhasil diperbarui\n", idCari);
8             return;
9         }
10        bantu = bantu->next;
11    }
12 }
```

```
1 // Hapus pasien tertentu berdasarkan namanya
2 void hapusTengah(char namaCari[]) {
3     if (head == NULL) return; // Kalau kosong ya nggak ada yang bisa dihapus
4
5     struct Node *bantu = head;
6     struct Node *sebelum = NULL;
7
8     // untuk mencari nama orang
9     while (bantu != NULL && strcmp(bantu->nama, namaCari) != 0) {
10         sebelum = bantu; // Simpan posisi sebelumnya
11         bantu = bantu->next; // Maju terus sampai ketemu atau habis
12     }
13
14     // Kalau namanya ketemu di list
15     if (bantu != NULL) {
16         // Kalau ternyata yang dihapus itu orang pertama
17         if (sebelum == NULL) {
18             head = head->next;
19         } else {
20             // "Lompatin" node yang dihapus: sambungin orang sebelumnya ke orang sesudahnya
21             sebelum->next = bantu->next;
22         }
23         free(bantu);
24     }
25 }
```

```
1 // nama : deden ahmad jamil
2 // nim : 2501518
3
4 #include "antrian_rs.h"
5
6 int main() {
7     // jawaban no 1
8     insertAwal(1, "Ryan", "Demam");
9     insertAkhir(2, "Reybano", "Batuk");
10    insertAkhir(3, "Akbar", "Flu");
11
12    // jawaban no 2
13    insertAwal(0, "Radit", "Sesak Napas");
14    insertTengahSebelum(3, 4, "Sasa", "Sakit Kepala");
15
16    // jawaban no 3
17    updatePenyakit(1, "Tipes");
18    insertTengahSesudah(4, 5, "Aura", "Maag");
19    hapusAwal();
20    hapusAkhir();
21
22    // jawaban no 4
23    updateNama(4, "Sasha");
24    updatePenyakit(2, "Radang Tenggorokan");
25    insertAkhir(6, "Galuh", "Luka Bakar");
26    hapusTengah("Aura");
27
28    // tampilkan hasil
29    tampilkan();
30
31    return 0;
32 }
```

Ss output nomor 4:

```
deden@Dden MINGW64 ~/OneDrive/Dokumen/matakuliah(semester2)/strukdat/TUGAS_LATIHAN_PRAKTIKUM/LP4-linkedlist-part2/no4
$ ./LAT2_A_DEDEN_2501518.exe
Data pasien Ryan berhasil diperbarui.
Data pasien dengan id 4 berhasil diperbarui
Data pasien Reybano berhasil diperbarui.

Data Antrian Pasien:
=====
ID: 1 | Nama: Ryan | Penyakit: Tipes
ID: 2 | Nama: Reybano | Penyakit: Radang Tenggorokan
ID: 4 | Nama: Sasha | Penyakit: Sakit Kepala
ID: 6 | Nama: Galuh | Penyakit: Luka Bakar
=====
```

2. Kesimpulan

Kesimpulan

Jujur, setelah ngulik soal ini aku baru ngerasa kalau *Linked List* itu ternyata jauh lebih fleksibel daripada *array* karena kita bisa bebas nambah atau hapus data pasien tanpa pusing mikirin batas memori. Serunya (sekaligus deg-degannya) itu pas mainin *pointer*; aku harus bener-bener teliti pas nyelipin atau hapus data di tengah antrean, karena kalau salah sambung dikit aja pake *pointer* bantuan kayak bantu atau sebelum, rantainya bisa putus dan datanya hilang. Walaupun pas mau *update* nama atau penyakit aku harus sabar nyisir antrean satu-satu dari depan, tapi ini bikin aku makin dapet *feel* nya gimana cara kerja memori dinamis di bahasa C yang sebenarnya